Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Dersin tarihi 2 Mayıs 2023 Salı. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:01'de başlar. Hoca bunun dönemin 10. haftası olduğunu söyledi.
  - Ders iki kez kısa süre bölündü (bölümden telefon). Ders yaklaşık 70 dakika sürdü.
- Vize sonuçları ve final bilgisi
- Dersin geri kalanının planı
  - Tasarım aşamasının son konusu: veri tasarımı.
  - Sonraki konular: gerçekleştirme (kodlama), test, sisteme geçiş (kurulum) ve bakım.
- Veritabanı tasarım süreci
  - Gereksinim analizi, kavramsal ve mantıksal tasarım, normalizasyon, veritabanı uyarlaması (fiziksel gerçekleştirme).
- Veri modelleri
  - Hiyerarşik (sıra düzensel) model, XML ve JSON (yarı yapılı), nesne modeli ve ORM, NoSQL ve bellek içi veritabanları, varlık-ilişki modeli.
- Varlık-ilişki (Entity-Relationship, ER) modeli
  - Semboller: varlık, öznitelik, anahtar öznitelik, ilişki.
  - İlişki türleri: bire bir (1:1), bire çok (1:N), çoka çok (N:M).
  - Örnekler: personel-departman, kitapçı veritabanı, film-sinema.
  - Zayıf varlık (weak entity).
- ER modelinin tablolara dönüştürülmesi: birincil anahtar (primary key) ve yabancı anahtar (foreign key).
- Proje raporu beklentileri.

Hocanın Özellikle Vurguladığı Kısımlar

- Vize testti ve sistem otomatik notlandırdı. Bir öğrenciye göre ortalama 82.
  - Final ağırlığı değişmeyecek, %40 kalacak.
  - Finaller yüz yüze yapılacak.
- Süreçte bir hata ne kadar erken yapılır ve ne kadar geç fark edilirse maliyeti o kadar yüksek olur.
- Bakım aşaması bir yazılımın kullanıldığı bütün ömür boyunca sürer. En maliyetli, ama aynı zamanda en çok para kazandıran aşamadır.
- Bu derste odak ilişkisel veritabanı ve varlık-ilişki modelidir.
  - Normalizasyon ayrıntıları Veritabanı Yönetim Sistemleri dersinde işlenecek.
- Bilgi sistemlerinde yalnız NoSQL veya yalnız MongoDB kullanmak zorunda değilsiniz.
  - Büyük ve zamanla değişen veri için yeni çözümler, iş mantığı verisi için ilişkisel veritabanı birlikte kullanılabilir.
- Veritabanı katmanında kod (saklı yordam) yazmak eskiden performans için yapılırdı. Günümüzde kavramların ayrı tutulması ilkesi gereği pek sevilmiyor.
- Birincil anahtarlar değişmez. Ad gibi diğer alanlar değişebilir. Örnek: Bölümün adı "Bilgisayar Bilimleri ve Mühendisliği"nden "Bilgisayar Mühendisliği"ne değişse de kodu aynı kalır ve veri tutarlılığı korunur.
- Proje raporunda yalnız diyagram koyup geçmeyin. Sonuçlara nasıl ulaşıldığını anlatın.

Kısa Tekrar Notları

- Veritabanı tasarım adımları: gereksinim analizi, kavramsal tasarım (ER modeli), mantıksal tasarım, normalizasyon, fiziksel uyarlama.
- Normal formlar: 1NF, 2NF, 3NF, Boyce-Codd normal formu (BCNF). Amaç veri tekrarını önlemek ve tutarlılığı sağlamaktır.
- ER sembolleri:
  - Dikdörtgen: varlık (varlık kümesi).
  - Oval: öznitelik.
  - Altı çizili oval: anahtar öznitelik (birincil anahtar).
  - Eşkenar dörtgen: ilişki (bağıntı).
  - Çift çizgili dikdörtgen: zayıf varlık.
- Veritabanı karşılıkları:
  - Varlık kümesi = tablo.
  - Varlık = tablodaki bir satır.
  - Öznitelik = tablodaki bir sütun.
- İlişkinin kendi öznitelikleri olabilir. Örnek: "çalışır" ilişkisinde "başlangıç tarihi".
- İlişki türleri:
  - 1:1 örnek: evlilik (bir erkek - bir kadın).
  - 1:N örnek: bir bölümde çok öğrenci okur, bir öğrenci tek bölümde okur.
  - N:M örnek: bir öğrenci çok ders alır, bir derste çok öğrenci vardır.
- Tablolara dönüştürme:
  - 1:1: taraflardan birinin birincil anahtarı, diğerine yabancı anahtar olarak eklenir.
  - 1:N: "1" tarafının birincil anahtarı, "N" tarafındaki tabloya yabancı anahtar olarak eklenir.
  - N:M: ilişki için ayrı bir tablo açılır. İki tarafın birincil anahtarları bu tabloya yabancı anahtar olarak eklenir.
  - Zayıf varlık: sahibi olan varlığın birincil anahtarı eklenir. İki alan birlikte birincil anahtar olur.
- Dersin kalan planı: 11. hafta vaka çalışması, 12. hafta son teorik ders, 13-14. haftalar proje sunumları.

Detaylı Açıklamalar

Dersin kalan planı ve süreç

Tasarım aşamasında son konu veri tasarımıdır. Sonra süreç şöyle devam eder:

1. **Gerçekleştirme:** kodlamada nelere dikkat edilmeli, nereden başlanmalı, kod kalitesi.
2. **Test:** hangi testler neden yapılır, sürecin hangi kısmını karşılar. Hata bulunursa geri dönülür:
   - Kodda hata varsa düzeltilip tekrar test edilir.
   - Tasarım yanlışsa iki adım geri gidilip tasarım güncellenir.
   - Problem yanlış anlaşıldıysa analize kadar geri gidilir.
3. **Kurulum ve geçiş:** yeni sistem nasıl devreye alınır, mevcut sistemden nasıl geçilir.
4. **Bakım:** sistem bir canlı organizma gibi kullanım ömrü boyunca bakımla yaşatılır. Bakım türleri ve neyi değiştirdikleri işlenecek.

Veritabanı tasarım süreci

Veritabanı tasarımı, bilgi sistemi geliştirme süreciyle paralel yürür:

- **Gereksinim analizi ve fizibilite:** hangi verilerin tutulacağı ve gereken yazılım ile donanımın maliyetleri belirlenmeye başlanır.
- **Kavramsal ve mantıksal tasarım:** hangi verilerin tutulacağı ve aralarındaki ilişkiler belirlenir. Varlık-ilişki modeli oluşturulur. Ayrıntılı tasarımda mantıksal model bütün yönleriyle hazırlanır.
- **Normalizasyon:** veri tekrarını önlemek, tutarlılığı sağlamak ve performansı karşılamak için mantıksal modele uygulanır. Bu derste ayrıntısına girilmedi.
- **Veritabanı uyarlaması:** veritabanı yönetim sistemi kurulur ve ayarlanır. Tablolar, kısıtlar oluşturulur ve ilk veriler yüklenir.

- **Veritabanı programlama:** bazı işlevler veri katmanında fonksiyon ve yordamlarla (procedure) yazılabilir. Eskiden performans için çok yapılırdı. Büyük veri döneminde ilişkisel modellerde bu artık anlamsız kaldı. Kavramların ayrı tutulması ilkesi de buna karşıdır.

Veri modelleri

- **Hiyerarşik (sıra düzensel) model:** Hoca bunu dosya sistemi üzerinde kayıt (record) tutan kayıt yöneticileriyle anlattı. C'deki `struct`'ların dosyada tutulması gibi. Günümüzde pek kullanılmıyor.
- **XML (Extensible Markup Language):** yarı yapılı veri modeli. Veri ve veriyi tanımlayan üst veri (metadata) bir aradadır. Daha çok farklı platformlar arasında veri alışverişi için kullanılır. 15-20 yıl önce endüstri standardıydı. Android manifest dosyaları da XML'dir. Ek yükü (overhead) fazladır: doğrulama ve stil dosyaları da gerekir.
- **JSON (JavaScript Object Notation):** XML'in daha hafif bir karşılığı. Bugün heterojen sistemler arasında veri aktarımında neredeyse standart.
- **Nesne modeli:** uygulamadaki nesnelerin doğrudan nesne veritabanında saklanması. Çok tutmadı, çünkü işlevsel çözümlemenin de sürdüğü alanlar (ör. gömülü sistemler) var ve ilişkisel veritabanları da güçlendi.
- **ORM (Object Relational Mapping):** uygulama nesneye yönelik geliştirilir, arkada ilişkisel veritabanı kullanılır. Aradaki dönüşümü bir ara katman yapar. Örnek: Hibernate.
- **NoSQL ve bellek içi veritabanları:** MongoDB, AllegroGraph, Redis gibi. Video gibi büyük ve hacimli verilerde ilişkisel veritabanı çok pahalıya gelir (ör. kapasiteye göre lisans). Bu sistemler ilişkisel modelin alternatifi değil, destekçisi olarak kullanılır.
- **Varlık-ilişki modeli:** en yaygın model. Bu dersin odağı.

Varlık, öznitelik ve anahtar

- **Varlık:** bilgi sisteminde var olan ve benzerlerinden ayırt edilen her şey. Örnek: kitap, öğrenci, araba. Nesneye yönelik analizdeki kavramsal sınıflar da veri tasarımında varlık olarak modellenebilir.
- **Varlık kümesi:** aynı türdeki varlıkların tamamı. Veritabanında bir tablodur. Örnek: bütün kitaplar = kitap tablosu.
- **Öznitelik:** varlığın tutulması gereken özelliği. Öğrenci için ad, sınıf, numara, ağırlıklı not ortalaması, bölüm. Tablodaki bir sütundur.
- **Anahtar öznitelik (birincil anahtar):** değeri kümedeki her varlık için benzersiz olan öznitelik. Altı çizilerek gösterilir.
  - Örnek: Personel (sicil no, ad, görev, maaş). Ad, görev ve maaş aynı olabilir. Sicil no benzersizdir ve anahtardır.

Bir varlıkta yalnız sistemin kullanacağı öznitelikler tutulur. Örnek: personelin boyu ve kilosu sistemde kullanılmıyorsa modelde yer almaz.

İlişkiler

İki varlık kümesi arasındaki bağa ilişki (bağıntı) denir. Eşkenar dörtgenle gösterilir ve içine adı yazılır.

Personel-departman örneği: Personel (sicil no, isim, kazanç) ve departman (numara, ad, bütçe) varlıkları "çalışır" ilişkisiyle bağlanır. Bir personel bir veya birden fazla departmanda çalışabilir. Bu kararı sistemin ihtiyacı belirler. Bir departmanda genelde çok personel vardır. Hiç personeli olmayan departman da olabilir (sıfır veya daha fazla).

İlişkinin kendi öznitelikleri olabilir. Hoca kendi adıyla örnek verdi: Göksel, Bilgisayar Mühendisliği'nde 2003'ten beri çalışıyor. Başka bir bölüme geçerse oradaki başlangıç tarihi farklı olur. Bu tarih ne personele ne departmana aittir, "çalışır" ilişkisine aittir.

İlişki türleri:

- **Bire bir (1:1):** bir kümedeki her varlık, diğer kümede en çok bir varlıkla ilişkilidir. Örnek: evlilik. Az karşılaşılır.

- **Bire çok (1:N):** “1” tarafındaki bir varlık, “N” tarafında birçok varlıkla ilişkili olabilir. “N” tarafındaki bir varlık ise en çok bir varlıkla ilişkilidir. Örnek: çift ana dal yoksa bir bölümde çok öğrenci okur, bir öğrenci tek bölümde okur.
- **Çoka çok (N:M):** iki taraf da diğer kümedeki birçok varlıkla ilişkili olabilir. Örnek: bir öğrenci BLM2042 ve BLM2512 Veri Yapıları’nı birlikte alabilir. Bir derste de 80-90 öğrenci vardır.

Kitapçı veritabanı örneği

Slayttaki ER diyagramı:

- **KİTAP:** ISBN (anahtar), Başlık, YayıneviID, Fiyat.
- **YAZAR:** YazarID (anahtar), Yazar Adı, Yazar Soyadı, Yazar Telefon No.
- **YAYINEVİ:** YayıneviID (anahtar), Yayınevi Adı, Yayınevi Telefon No.
- KİTAP - YAZAR: “tarafından yazılmış” ilişkisi, çoka çok. Bir yazar çok kitap yazabilir. Bir kitabın birden çok yazarı olabilir.
- KİTAP - YAYINEVİ: “yayımcısı olma” ilişkisi, bire çok. Bir yayınevi çok kitap basar. Bir kitap tek yayınevine aittir.

Soru: Dostoyevski’yi 50 yayınevi basmıyor mu? Basıyor. Ama her baskının ISBN numarası farklıdır. Kağıt, renk, font da farklıdır. Sistemde bunlar ayrı kitap varlıklarıdır. İstenirse nesneye yönelik tasarımdaki “ürün tanımı” gibi ayrı bir “kitap tanımı” varlığı da kurulabilir.

Kitaptaki YayıneviID özniteliği, ilişki diyagramda zaten gösterildiği için ER diyagramında yazılmak zorunda değildir. Hoca bunu fazladan koyduğunu söyledi. Tabloya dönüştürmede ise bu alan yabancı anahtar olarak yer alır:

1. ISBN’si bilinen kitaba benzersiz anahtarla erişilir.
2. Kitabın YayıneviID değeri okunur (ör. 345).
3. Yayınevi tablosunda ID’si 345 olan satırdan adı ve telefonu alınır.

Kitap ile yazar arasındaki çoka çok ilişki için ayrı bir tablo gerekir. Her satırda bir ISBN ve bir YazarID bulunur. Aynı ISBN için birden çok yazar, aynı yazar için birden çok kitap satırı olabilir. İstenirse yazar sırası gibi bir sütun da eklenebilir.

ER modelini tablolara dönüştürme

Varlık kümeleri tablolara, öznitelikler sütunlara dönüşür. İlişkilerin türüne göre anahtarlar eklenir.

Bire bir: taraflardan birinin birincil anahtarı, diğer tabloya yabancı anahtar olarak eklenir. Hangi tarafın seçileceği bağlama göre karar verilir.

Bire çok (slayttaki personel-bölüm örneği):

```
Personel (personelId, adSoyad, adres, maas, bolumId)   -- personelId: PK, bolumId: FK
Bolum    (bolumId, adi)                                -- bolumId: PK
```

bolumId	adi
011	BLM
012	EHM

personelId	adSoyad	…	bolumId
345	Göksel	…	011

“345 kodlu personel hangi bölümde çalışıyor?” sorusu: Personel tablosunda 345’e gidilir, bolumId 011 okunur. Bölüm tablosunda 011’in adı BLM’dir.

Çoka çok (film-sinema örneği):

- Film (ID, ad, yapım tarihi, yönetmen).

- Sinema (ID, ad, adres, telefon).
- Bir film çok sinemada oynar, bir sinemada çok film oynar.
- Araya "oynar" ilişki tablosu açılır: FilmID, SinemaID, tarih, saat.
- İki yabancı anahtar birlikte benzersiz tanımlayıcı olabilir. Ama genelde tabloya ayrı bir ID alanı eklenir. Veritabanı yönetim sistemleri bunu otomatik artan sayı olarak üretebildiği için bu yol daha hızlıdır.
- Örnek satırlar: 10 numaralı film AA ve BB sinemalarında, 11 numaralı film AA ve CC sinemalarında, farklı tarih ve saatlerde.
- "10 numaralı film hangi sinemalarda?" sorusu: ilişki tablosunda FilmID'si 10 olan satırlardan SinemaID'ler (AA, BB) alınır. Sonra sinema tablosundan adları ve telefonları okunur.

Zayıf varlık

Kendi başına var olamayan ve kendi öznitelikleriyle benzersiz tanımlanamayan varlıktır. Çift çizgili dikdörtgenle gösterilir.

Örnek: fakülte. "Mühendislik Fakültesi" birçok üniversitede vardır. Kodları ve diğer bilgileri aynı olabilir. Fakülte ancak bir üniversiteyle birlikte anlamlıdır. Bir üniversitede N fakülte vardır.

Tabloya dönüştürme: sahibi olan varlığın (üniversite) birincil anahtarı fakülte tablosuna eklenir. Üniversite no ve fakülte no birlikte birincil anahtar olur. Okunuşu: "şu üniversitedeki şu kodlu fakültenin adı nedir?"

Proje raporu beklentileri

Bir öğrenci rapor için şablon veya örnek istedi. Hoca örnek vermeyeceğini söyledi, çünkü örnek verilince herkes aynısını yazıyor. Ders duvarında asgari içerik listesi paylaşıldı. Beklentiler:

- **Fizibilite:** yalnız toplantı raporu, organizasyon şeması ve Gantt diyagramı koymak fizibilite değildir.
  - Teknik, ekonomik ve zaman fizibilitesi yapılmalı.
  - Yazılım fizibilitesinde seçilen geliştirme ortamı alternatifleriyle karşılaştırılmalı. "Java biliyoruz, Java seçtik" yeterli değil. .NET veya Python ile maliyet, eğitim süresi ve deneyim birlikte skorlanmalı.
  - Gantt diyagramının nasıl oluştuğu gösterilmeli: iş paketleri nasıl bulundu, alt faaliyetler ve aralarındaki ilişkiler neler, hangi kaynak ne kadar yükleniyor.
- **Sistem analizi:** iş analizi yapılıp belgelenmeli.
  - İşlevsel çözümlemede veri akış diyagramları ve süreç formları yazılmalı.
  - Nesneye yönelik çözümlemede kullanım senaryolarıyla başlanmalı. Kavramsal sınıfların nasıl bulunduğu anlatılmalı ve sınıf diyagramı gösterilmeli.

Duyurular

- 9 Mayıs'taki derste (11. hafta) hoca dersi başlatacak. Ardından bir vaka çalışması, yani örnek bir proje, tanıtılacak. Gantt diyagramı uygulamasında yardım eden kişiler bu uygulamayı da yapacak. İki grup birleşebilir. Gerekirse başka bir toplantı linki paylaşılacak.
- Vaka çalışması, öğrenciler projelerinin sonuna yaklaşırken örnek görsünler diye bir hafta öne alındı. Kodlama, test, geçiş ve bakım konuları 12. haftaya kaldı.
- 12. hafta son teorik ders. 13 ve 14. haftalarda proje ekipleri, hocanın açıklayacağı takvime göre sunum yapacak.

Ders sonrası notlar

- **Hiyerarşik model:** Hoca "sıra düzensel" modeli dosyalardaki kayıt yöneticileri ve `struct`'larla açıkladı. Hiyerarşik veri modelinin asıl özelliği, kayıtların ağaç yapısında ebeveyn-çocuk ilişkisiyle düzenlenmesidir (ör. IBM IMS). Düz dosyada kayıt tutmak tek başına hiyerarşik model değildir.
- **Bire çok tanımı:** Derste "bire çok ilişkide bir kümedeki her bir varlık diğer kümede en çok tek bir varlıkla ilintili olabilir" dendi. Bu cümle yalnız "N" tarafını anlatır. "1"

tarafındaki bir varlık, diğer kümede birçok varlıkla ilişkili olabilir. Özette iki taraf da yazıldı.

- **Zayıf varlığın ilişkisi:** Zayıf varlığı sahibine bağlayan tanımlayıcı ilişki (identifying relationship) genellikle çift çizgili eşkenar dörtgenle gösterilir. Zayıf varlığın kendi kısmi anahtarı da kesikli çizgiyle altı çizilerek gösterilir.
- **Yabancı anahtarın gösterimi:** Slayttaki `Personel (personelId, adSoyad, adres, maas, bolumId)` satırında bolumId de altı çizili. Birçok kaynakta yalnız birincil anahtarın altı düz çizilir. Yabancı anahtar ya hiç çizilmez ya da kesikli çizgi veya italikle gösterilir. Karışıklık olmaması için sınavda hangisinin PK, hangisinin FK olduğunu ayrıca belirtin.
- **"Yapılanmış bellek":** Altyazıda NoSQL veritabanlarıyla birlikte "yapılanmış bellek" ifadesi geçiyor. Kastedilen büyük ihtimalle bellek içi (in-memory) veya yapılandırılmamış veri için tasarlanmış veritabanlarıdır. Kayıttan kesin anlaşılmıyor.
- **"2024'ten beri":** Hoca ilişki özniteliği örneğinde başka bölüme geçişi varsayımsal olarak "2024'ten beri" diye anlattı. Ders 2023'te kaydedildi. Bu yalnız bir örnek.
- **Ortalama:** Vize ortalaması (82) bir öğrencinin OBS'den gördüğü değerdir. Hoca doğrulamadı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi daha verimli çalışabilirsiniz.